Reviewer Pack — Schur-Weyl Invariant Ratio for Random Boolean Functions

Entry e45b325ac803 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 00:46:54 UTC

Schur-Weyl Invariant Ratio for Random Boolean Functions

Entry ID: `e45b325ac803` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 00:46:54 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl duality and Hecke algebras **Field B** (complexity object): Complexity Theory: Circuit Complexity (Random Boolean Functions)

Statement:

> ['For every n ≥ 2, there exists a constant c(n) such that for a random boolean function f: {0,...,n-1} -> {0,1}, the ratio of Schur-Weyl invariants ρ(f) = ρ_s(f)/ρ_t(f), where ρ_s(f) is the sum of symmetric functions and ρ_t(f) is the sum of trace functions, satisfies ρ(f) ≥ c(n)', 'For a specific instance of size n, if there exist two random boolean functions f1 and f2 such that ρ(f1) < c(n) and ρ(f2) ≥ c(n), then there exists an embedding from f1 to f2 that preserves the Schur-Weyl invariants.']

Rationale (proposer's reasoning):

> ['The Schur-Weyl duality provides a bridge between linear algebra, combinatorics, and representation theory. By leveraging this duality, we can analyze properties of boolean functions through their representations. If the conjecture is true, it would imply that for certain random boolean functions, there exists an inherent complexity barrier that cannot be bypassed.', 'This conjecture connects the abstract concepts of Schur-Weyl invariants with the concrete notion of circuit complexity. It suggests that there might be a deep connection between the representation theory of boolean functions and their computational hardness.']

Taxonomy category: `GCT_DET_PERM` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `10b9932ad53ce9cc`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> We accept the conjecture if for n ≤ 40, at least 95% of randomly generated boolean functions f yield ρ(f) ≥ c(n) where c(n) > 1. We falsify the conjecture if any seed produces ρ(f) < c(n), or if c(n) is not significantly larger than 1 after analyzing multiple seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `(Schur-Weyl duality OR Hecke algebras) AND (random boolean functions OR circuit complexity)` - `(Schur-Weyl invariant ratio) AND (circuit complexity OR random Boolean Functions)` - `(embedding preserving Schur-Weyl invariants) AND (Boolean function complexity)`

Top relevant hits considered: - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras - [http://arxiv.org/abs/2603.00603v2] Mirabolic Hecke algebras, Schur-Weyl duality and Frobenius character formulas - [http://arxiv.org/abs/2006.13879v1] Two dualities: Markov and Schur-Weyl - [http://arxiv.org/abs/0808.0684v1] 9-variable Boolean Functions with Non-linearity 242 in the Generalized Rotation Class - [http://arxiv.org/abs/2309.15026v3] Instance complexity of Boolean functions - [http://arxiv.org/abs/1912.11518v2] Fluctuations of the spectrum in rotationally invariant random matrix ensembles - [http://arxiv.org/abs/1206.6075v1] Well-founded Boolean ultrapowers as large cardinal embeddings - [http://arxiv.org/abs/2211.16402v1] Query complexity of Boolean functions on slices

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def schur_weyl_invariant(f):
        n = len(f)
        rho_s = 0
        rho_t = 0
        for i in range(2**n):
            bitstring = format(i, f'0{n}b')
            value = int(bitstring, 2)
            if value == f(value):
                rho_s += 1
            rho_t += 1
        return Fraction(rho_s, rho_t)

    def random_boolean_function(n):
```

```
        return [random.choice([0, 1]) for _ in range(2**n)]

    n = random.randint(5, 40)
    f = random_boolean_function(n)
    rho_f = schur_weyl_invariant(f)

    if rho_f < Fraction(1, 2):
        return {
            "metric_name": "Schur-Weyl Invariant Ratio",
            "metric_value": float(rho_f),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"rho(f) = {rho_f} < c(n)"
        }

    return {
        "metric_name": "Schur-Weyl Invariant Ratio",
        "metric_value": float(rho_f),
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    if not sys.argv[1:]:
        seeds = [2**i + 1 for i in range(5, 6)]  # Using a small set of primes as default
    else:
        seeds = list(map(int, sys.argv[1:]))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"rho(f) < c(n)\" first_failing_seed={first_faili
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means we cannot confirm or falsify the conjecture based on the results. | next: Re-run the test with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15107
2	preregistration	ollama_remote	glm4:latest	0	0	5783
3	novelty	ollama_remote	glm4:latest	0	0	4877
4	novelty	ollama_remote	glm4:latest	0	0	6809
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12631
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6382
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7942
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7405
9	judge	ollama_remote	glm4:latest	0	0	23300

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 90235 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e45b325ac803.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e45b325ac803.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e45b325ac803.tar.gz` (if generated)